

TAG

Total Adoration & Global Intercession

Récapitulatif de construction — État du dépôt

7 AOÛT 2026

Ce document récapitule, en détail, tout ce qui a été effectivement construit dans le dépôt `tag-platform` jusqu'ici : la base de données réelle (Supabase), l'API NestJS, le début de l'application web React, ce qui a été vérifié par des tests réels, les décisions de simplification assumées, et ce qu'il reste à construire. L'objectif est de donner une sensibilité concrète du fonctionnement actuel de l'application — pas un plan théorique.

2

DOMAINES
COMPLETS

13

TABLES EN BASE

8

RÔLES
HIÉRARCHIQUES

17

PERMISSIONS
ATOMIQUES

~30

ROUTES API

Sommaire

1. Vision du produit (rappel)
2. Architecture technique
3. Domaine Identity (Auth) **CONSTRUIT**
4. Domaine Community **CONSTRUIT**
5. Application web
6. Ce qui a été vérifié
7. Décisions de simplification assumées
8. Ce qui reste à construire
9. Comment tester aujourd'hui

1. Vision du produit (rappel)

TAG est une plateforme mondiale d'intercession chrétienne continue : une salle de prière est active quelque part dans le monde à chaque instant, 24h/24, 7j/7. Le programme est découpé en créneaux successifs (`PrayerSlot`), animés par un responsable humain ou, en son absence, par une IA Animatrice. L'IA anime et assiste, mais ne remplace jamais l'autorité pastorale sur les décisions sensibles.

Le dépôt suit un ensemble de 17 documents de spécification (`docs/01` à `docs/17`) qui font autorité sur toute décision d'architecture ou de règle métier. Rien n'est censé être inventé en dehors de ce qu'ils décrivent.

2. Architecture technique

Couche	Choix effectif
Monorepo	Nx Workspace, npm workspaces (<code>packages/api</code> , <code>packages/web</code>)
Backend	NestJS 11, TypeScript strict, validation via <code>class-validator</code>
Base de données	Supabase (PostgreSQL réel, projet en ligne) — le serveur API se connecte avec la clé de service, contourne RLS
Frontend web	React 19 + Vite + react-router-dom (le document de roadmap recommandait Angular ; le dépôt existant avait déjà React — poursuivi par cohérence avec l'existant plutôt que réécrit)
Authentification	JWT (access token) + refresh token opaque haché, maison — pas de dépendance à Supabase Auth
Documentation API	Swagger généré automatiquement, servi sur <code>/api/docs</code>

Convention de code observée dans tout le dépôt : les services accèdent directement au client Supabase (pas de couche ORM), les contrôleurs restent minces (validation → appel service → retour), les DTO d'entrée sont en `camelCase` , les entités/réponses reflètent les colonnes de la base en `snake_case` .

3. Domaine Identity (Auth) CONSTRUIT ET VÉRIFIÉ

Fondation de toute l'application — premier domaine construit, conforme à l'ordre de génération imposé par les règles du projet.

3.1 Base de données

7 tables créées : `users` , `roles` , `permissions` , `role_permissions` , `role_assignments` , `refresh_tokens` , `password_reset_tokens` .

- **8 rôles hiérarchiques** : Visiteur → Nouveau Converti → Intercesseur → Modérateur → Responsable d'Équipe → Pasteur → Administrateur → Super Administrateur.
- **17 permissions atomiques** (ex. `prayer_request.create` , `testimony.approve` , `user.manage`) réparties sur **73 associations rôle-permission**, reflétant exactement la matrice du document RBAC.
- Mots de passe hachés en **Argon2id** ; politique : 12 caractères minimum, majuscule, minuscule, chiffre, caractère spécial.

3.2 Sécurité des sessions

- Access token JWT, durée de vie 15 minutes.
- Refresh token opaque (aléatoire, 48 octets), jamais stocké en clair — seul son empreinte SHA-256 est en base.
- **Rotation** : chaque rafraîchissement invalide l'ancien token et en émet un nouveau.
- **Détection de réutilisation** : si un refresh token déjà révoqué est présenté à nouveau (signe de vol), toute la famille de sessions de l'utilisateur est révoquée immédiatement.
- Changement de mot de passe (réinitialisation) révoque automatiquement toutes les sessions actives.

3.3 RBAC — contrôle d'accès

Chaque route sensible vérifie une **permission atomique** (jamais un nom de rôle en dur) via un Guard NestJS dédié. Point clé validé par des tests réels :

Portée globale vs. scopée à une communauté. Une attribution de rôle globale (`community_id = NULL`) donne ses permissions partout. Une attribution scopée à une communauté précise ne donne ses permissions *que* dans cette communauté — vérifié en direct : un utilisateur promu Responsable d'Équipe uniquement sur une cellule ne peut pas agir sur une autre communauté (403 confirmé).

Anti-escalade : l'endpoint d'attribution de rôle plafonne volontairement à *Pasteur* — impossible d'attribuer Administrateur ou Super Administrateur par cette voie, conformément à la note du document RBAC.

3.4 Routes construites

Route	Description
POST /auth/register	Inscription, attribue automatiquement le rôle Visiteur, retourne les tokens
POST /auth/login	Connexion
POST /auth/refresh	Rotation du refresh token
POST /auth/logout	Révocation du refresh token
POST /auth/password/forgot / reset	Réinitialisation (envoi via un service mail console en attendant le domaine Communication)
GET / PATCH /users/me	Profil courant
GET /users	Liste paginée, réservée à user.manage
GET /users/:id	Soi-même ou user.manage
PATCH /users/:id/status , DELETE /users/:id	Administration (suspension, suppression douce)
GET /roles , POST /roles/assign / revoke	Gestion des rôles

4. Domaine Community CONSTRUIT ET VÉRIFIÉ

Deuxième domaine, dans l'ordre de génération imposé (Identity → Community → Prayer...).

4.1 Base de données

- `communities` — hiérarchie via `parent_id`, types `GROUP / TEAM / CELL / COUNTRY / CITY / CHURCH / MINISTRY`.
- `community_members`, `posts`, `comments`.
- Nouvelle permission `community.manage` ajoutée (extension explicitement autorisée par le document RBAC, qui prévoit que la table des permissions grandisse avec chaque nouvelle fonctionnalité) — accordée à Responsable d'Équipe et au-dessus.

4.2 Règles de fonctionnement

- Le créateur d'une communauté en devient automatiquement membre.
- Seuls les membres d'une communauté peuvent y publier ou y commenter (vérifié : un non-membre reçoit 403, devient autorisé après ajout comme membre).
- Créer une communauté à la racine (sans parent) exige une attribution *globale* de `community.manage` — créer une sous-communauté exige cette permission dans le contexte du parent.

4.3 Routes construites

Route	Description
<code>POST/GET/PATCH/DELETE /communities</code>	Cycle de vie de la communauté
<code>POST/DELETE/GET /communities/:id/members</code>	Gestion des membres
<code>POST/GET /communities/:id/posts</code>	Fil d'actualité de la communauté
<code>POST/GET /posts/:id/comments</code>	Commentaires

Les listes (utilisateurs, communautés, membres, posts, commentaires) renvoient toutes une pagination cohérente `{ data, meta: { page, limit, total, totalPages } }`.

5. Application web (React)

Amorce de l'interface, branchée sur la vraie API :

- Pages **Connexion** et **Inscription**.
- `AuthContext` : persistance de session (jetons en `localStorage`), restauration automatique au chargement, rafraîchissement silencieux si l'access token a expiré.
- Tableau de bord** protégé (redirection vers la connexion si non authentifié), affichant l'utilisateur courant.

Les deux maquettes visuelles statiques (salle de prière mondiale, écrans demandes/témoignages/calendrier/carte) existent séparément (`prayer-room-preview.html`, `tag-screens-preview.html`) mais ne sont pas encore

branchées à l'API réelle.

6. Ce qui a été vérifié

- **Build, lint, typecheck** propres sur `api` et `web` via Nx, à chaque étape.
- **Tests de bout en bout réels** (pas de mocks) contre le vrai projet Supabase, via des appels HTTP directs au serveur en cours d'exécution :
 - Inscription → connexion → route protégée → 401 sans jeton.
 - Rotation du refresh token, puis réutilisation de l'ancien jeton → 401 + révocation de toute la session.
 - RBAC : 403 pour un Visiteur, 200 après promotion en rôle habilité.
 - Plafond anti-escalade sur l'attribution de rôle.
 - Communautés : création refusée/autorisée selon permission et portée, adhésion, publication bloquée puis débloquée après adhésion, isolation de portée entre deux communautés.
- Toutes les données de test créées pendant ces vérifications ont été supprimées de la base après coup.

7. Décisions de simplification assumées

Choix délibérés, à connaître avant de continuer :

Sujet	Décision	Raison
<code>items</code> / <code>tags</code>	Non touchés	Scaffolding par défaut de Nx, absent des spécifications TAG — n'appartient pas au produit réel
Enveloppe <code>{success, data, meta}</code>	Appliquée uniquement aux listes paginées, pas à toutes les routes	Cohérence avec le code déjà existant plutôt qu'avec la spec API au pied de la lettre
MFA	Différé	Explicitement hors MVP selon la feuille de route
Édition/suppression de posts et commentaires	Non construites	Absentes de la liste de routes de la spécification API — rien n'est inventé au-delà du document
Envoi d'e-mail (mot de passe oublié)	Implémentation console (journalisée, pas envoyée)	Le domaine Communication (fournisseur mail réel) n'existe pas encore

8. Ce qui reste à construire

Dans l'ordre de génération du projet (chaque domaine dépend des précédents) :

1. **SUIVANT Prayer** — `PrayerProgram` / `PrayerSlot`, chronomètre serveur, transition automatique, salle mondiale continue, demandes de prière, témoignages.
2. **À FAIRE Communication** — notifications, e-mail réel, WhatsApp, publications multicanal.
3. **À FAIRE Storage** — fichiers, upload présigné.
4. **À FAIRE Analytics** — tableaux de bord, carte mondiale en temps réel.
5. **À FAIRE Administration** — journaux d'audit, sauvegardes.
6. **À FAIRE Agents IA** — Accueil, Intercession, Évangélisation, Modératrice, Communication, Analyse (dont « Le Veilleur » sur WhatsApp).
7. **À FAIRE Application mobile** Flutter, **événements, gamification**.
8. **À FAIRE Déploiement** Docker / production.

9. Comment tester aujourd'hui

Service	Adresse locale
Application web	<code>http://localhost:4200</code>
API	<code>http://localhost:3000/api</code>
Documentation Swagger	<code>http://localhost:3000/api/docs</code>

Démarrage : `npx nx serve api` et `npx nx serve web` à la racine du dépôt (deux terminaux séparés).

Document généré à partir de l'état réel du dépôt `tag-platform` (branche `master`) au 7 août 2026 — pas un plan théorique, un état des lieux vérifié.